

Load the rela datssets of the apple datasets

```
In [57]: # pip install yfinance
import warnings
warnings.filterwarnings('ignore')
import yfinance as yf

# Download Apple stock data
df = yf.download("AAPL", start="2020-01-01", end="2025-01-01")
```

```
[*****100%*****] 1 of 1 completed
```

```
In [58]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
import numpy as np

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
In [3]: # !pip install yfinance
```

```
In [59]: df.shape
```

```
Out[59]: (1258, 5)
```

```
In [60]: df
```

Out[60]:

Price	Close	High	Low	Open	Volume
Ticker	AAPL	AAPL	AAPL	AAPL	AAPL
Date					
2020-01-02	72.333878	72.394086	71.091184	71.344054	135480400
2020-01-03	71.630653	72.389273	71.406681	71.563221	146322800
2020-01-06	72.201408	72.239942	70.503546	70.754014	118387200
2020-01-07	71.861847	72.466330	71.642689	72.211049	108872000
2020-01-08	73.017838	73.318877	71.565621	71.565621	132079200
...	...	...	...	...	...
2024-12-24	256.560852	256.570768	253.669307	253.868050	23234700
2024-12-26	257.375610	258.448771	255.994450	256.550893	27237100
2024-12-27	253.967392	257.057664	251.453455	256.193162	42355300
2024-12-30	250.598907	251.890657	249.158116	250.628716	35557500
2024-12-31	248.830215	251.672059	247.846495	250.837396	39480700

1258 rows × 5 columns

```
In [61]: df.columns

Out[61]: MultiIndex([( 'Close', 'AAPL'),
                      ( 'High', 'AAPL'),
                      ( 'Low', 'AAPL'),
                      ( 'Open', 'AAPL'),
                      ('Volume', 'AAPL')],
                    names=['Price', 'Ticker'])

In [62]: columns=[item[0] for item in list(df.columns)]

In [12]: columns

Out[12]: ['Close', 'High', 'Low', 'Open', 'Volume']

In [63]: df.columns=columns

In [64]: df
```

Out[64]:

	Close	High	Low	Open	Volume
Date					
2020-01-02	72.333878	72.394086	71.091184	71.344054	135480400
2020-01-03	71.630653	72.389273	71.406681	71.563221	146322800
2020-01-06	72.201408	72.239942	70.503546	70.754014	118387200
2020-01-07	71.861847	72.466330	71.642689	72.211049	108872000
2020-01-08	73.017838	73.318877	71.565621	71.565621	132079200
...	...	...	...	...	...
2024-12-24	256.560852	256.570768	253.669307	253.868050	23234700
2024-12-26	257.375610	258.448771	255.994450	256.550893	27237100
2024-12-27	253.967392	257.057664	251.453455	256.193162	42355300
2024-12-30	250.598907	251.890657	249.158116	250.628716	35557500
2024-12-31	248.830215	251.672059	247.846495	250.837396	39480700

1258 rows × 5 columns

In [65]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1258 entries, 2020-01-02 to 2024-12-31
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   Close   1258 non-null   float64
 1   High    1258 non-null   float64
 2   Low     1258 non-null   float64
 3   Open    1258 non-null   float64
 4   Volume  1258 non-null   int64  
dtypes: float64(4), int64(1)
memory usage: 59.0 KB
```

In [18]: `print(df.isnull().sum())`

```
Close      0
High       0
Low        0
Open       0
Volume     0
dtype: int64
```

In [16]: `print(df.isnull().sum())`

Close 0
High 0
Low 0
Open 0
Volume 0
dtype: int64

```
In [66]: X = df[["Open", "High", "Low", "Volume"]]  
        y = df["Close"]
```

```
In [67]: X
```

Out[67]:

	Open	High	Low	Volume
Date				
2020-01-02	71.344054	72.394086	71.091184	135480400
2020-01-03	71.563221	72.389273	71.406681	146322800
2020-01-06	70.754014	72.239942	70.503546	118387200
2020-01-07	72.211049	72.466330	71.642689	108872000
2020-01-08	71.565621	73.318877	71.565621	132079200
...	...	...	...	...
2024-12-24	253.868050	256.570768	253.669307	23234700
2024-12-26	256.550893	258.448771	255.994450	27237100
2024-12-27	256.193162	257.057664	251.453455	42355300
2024-12-30	250.628716	251.890657	249.158116	35557500
2024-12-31	250.837396	251.672059	247.846495	39480700

1258 rows × 4 columns

```
In [68]: y = df["Close"]  
        y
```

Out[68]:

Date	
2020-01-02	72.333878
2020-01-03	71.630653
2020-01-06	72.201408
2020-01-07	71.861847
2020-01-08	73.017838
...	
2024-12-24	256.560852
2024-12-26	257.375610
2024-12-27	253.967392
2024-12-30	250.598907
2024-12-31	248.830215

Name: Close, Length: 1258, dtype: float64

Train Testst Splits `

```
In [69]: X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2,random_state=
print("\nTRAIN SHAPE:", X_train.shape)
print("TEST SHAPE:", X_test.shape)
```

TRAIN SHAPE: (1006, 4)

TEST SHAPE: (252, 4)

Build Th einear Regression Models

```
In [70]: model = LinearRegression()
model.fit(X_train, y_train)

print("MODEL COEFFICIENTS")

coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})
coefficients
```

MODEL COEFFICIENTS

```
Out[70]:
```

	Feature	Coefficient
0	Open	-5.938475e-01
1	High	8.398486e-01
2	Low	7.516490e-01
3	Volume	-2.215775e-09

```
In [71]: X_test
```

Out[71]:

	Open	High	Low	Volume
Date				
2022-03-24	167.447673	170.462633	166.615631	90131400
2020-05-28	76.676406	78.290929	76.400464	133560800
2020-03-17	59.749788	62.187961	57.550601	324056000
2020-04-02	58.018916	59.180066	57.188487	165934000
2024-04-09	167.024648	168.390948	166.678133	42373800
...	...	...	...	...
2020-08-14	111.382080	111.546974	109.650674	165565200
2022-03-08	155.466128	159.440388	152.509898	131148300
2021-08-02	142.664705	143.239805	141.582730	62880000
2020-12-17	125.245291	125.906018	124.409674	94359800
2023-04-26	160.583522	162.769807	160.327477	45498800

252 rows × 4 columns

In [72]:

```
print("INTERCEPT")
print(model.intercept_)

y_pred = model.predict(X_test)
```

```
INTERCEPT
0.4550213593269632
```

In [73]:

```
y_pred
```

```
Out[73]: array([169.21619963,  77.8038464 ,  59.74104598,  58.32092366,
167.88030895, 151.02368904, 146.49518832, 179.35548101,
223.21258429, 180.75458699, 169.18570315, 167.64478623,
117.85967242, 151.07199112, 169.11963784, 121.32639358,
164.76203828, 133.42133762, 142.69422301, 189.34543047,
186.88004221, 145.17027218, 115.24933481, 130.28856336,
169.8079184 , 148.82055459, 151.39661829, 124.58785626,
164.99572776,  71.28261582, 169.12711554, 129.15984843,
225.85145605, 189.98618909, 146.19764907, 160.68866334,
117.54799327, 114.11814656, 147.67896487, 119.70633962,
219.73121348, 135.53701547, 171.39148734,  86.92615863,
163.35743762, 178.58547136, 135.340681 ,  77.34611977,
174.33556503, 170.33407959, 140.61651944, 127.16964016,
158.50429026, 190.88896674, 119.27127816, 123.07762898,
108.8990081 , 110.90631274, 230.30248614, 133.53020837,
164.03584856, 119.86677283, 144.70758444,  69.573273 ,
  61.7037104 , 224.31273202, 215.44952149,  68.07690739,
146.91473923,  64.30937694, 175.55866975,  66.70519728,
131.51579032, 189.20302118, 169.39617757, 194.45569202,
135.4799524 , 145.19409651, 144.4120685 , 245.76864096,
130.15699665, 124.38374875, 168.58664373, 117.62452824,
172.11575786, 202.84285858, 187.71456301, 162.95537787,
179.37215215, 247.26202944, 154.1319603 , 187.32307757,
170.59183523, 192.50911441,  69.51814133, 111.67002114,
120.86475892,  78.58094522,  76.82111115, 132.30480632,
179.13371165, 150.29827102, 186.49494761,  72.77641537,
132.53175771, 141.44674959, 185.94875803, 150.2079466 ,
234.5340583 , 139.35555118, 162.84621028, 142.74790644,
  90.93696268, 212.93777428, 118.75182155, 159.12154639,
128.62542452, 223.3535881 , 117.09899331, 175.61752536,
  68.77589571, 193.68714191, 211.96738086,  83.02836775,
189.43912874, 154.25369802, 145.10274193, 156.87998148,
122.49153312, 229.63180389, 251.15503872, 116.19217094,
  79.92193314, 180.34247749, 192.95601953, 152.89151131,
159.78433002, 151.14093022, 109.88621464, 146.20667069,
184.78477558, 137.53589792, 226.85594186, 179.82024566,
140.6152926 , 226.07313809, 222.23878381, 175.2558708 ,
194.99036403, 134.5264424 , 162.91590296,  56.55738552,
152.90417979, 156.70570021,  58.66525384,  83.05028328,
170.52106738, 136.82975123, 118.02455218, 216.27693956,
124.13319729, 126.58318524, 140.27008233, 146.20354238,
164.06665937, 153.71563985, 215.82110761, 119.59412591,
181.05339012, 120.38075806, 250.06744111, 145.86579225,
152.65276692, 162.20088444, 139.14480827, 180.9706108 ,
131.00383782, 112.46180782, 157.75378227, 119.59315065,
126.03024187, 167.98472278, 153.72132583, 129.13943507,
156.16903501, 169.01803441, 139.81249357, 135.39562672,
175.45990028, 105.82802684, 145.46040026, 143.76419024,
166.31320138, 135.4711801 , 148.01002255,  75.7141423 ,
106.85977515, 176.69083838, 180.8372766 , 140.73817112,
218.53806937, 228.83648572, 157.31729813, 131.96160082,
122.43723397, 119.93258758, 146.56331661, 176.01035928,
142.47556594, 141.35333685, 115.09611968, 172.120676 ,
144.95240783,  93.69139279, 118.96143357, 160.96254539,
156.76639726, 126.81493527, 134.09641461, 170.774257 ,
143.22590924, 111.96978731,  90.1488225 , 167.1358194 ,
```

```
167.40605851, 142.75610883, 222.94471977, 170.19279175,
186.79323479, 170.14497683, 62.63332802, 141.64294314,
130.33990831, 124.09625207, 166.40355917, 245.06325966,
182.42915142, 77.23065441, 105.87011554, 117.47922844,
127.58310324, 119.84915721, 171.36206744, 224.80455877,
116.65656968, 160.33853962, 78.24089949, 110.04558154,
156.38094896, 142.31487709, 125.12373448, 162.20405984])
```

In [74]: `y_test`

Out[74]:

Date	
2022-03-24	170.394119
2020-05-28	77.034653
2020-03-17	61.041298
2020-04-02	59.126957
2024-04-09	167.985016
	...
2020-08-14	111.457253
2022-03-08	154.115265
2021-08-02	141.845917
2020-12-17	125.050964
2023-04-26	161.272888

Name: Close, Length: 252, dtype: float64

Evaluats the Models

In [75]:

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("MODEL PERFORMANCE")

print("MAE :", mae)
print("MSE :", mse)
print("RMSE :", rmse)
print("R2 SCORE :", r2)
```

```
MODEL PERFORMANCE
MAE : 0.7418783265214773
MSE : 0.9964426015834871
RMSE : 0.9982197160863369
R2 SCORE : 0.9994242481621672
```

In [76]:

```
results = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

print("\nACTUAL VS PREDICTED")
results.head(10)
```

ACTUAL VS PREDICTED

Out[76]:

	Actual Price	Predicted Price
0	170.394119	169.216200
1	77.034653	77.803846
2	61.041298	59.741046
3	59.126957	58.320924
4	167.985016	167.880309
5	151.420288	151.023689
6	145.673431	146.495188
7	179.618317	179.355481
8	222.383606	223.212584
9	182.034103	180.754587

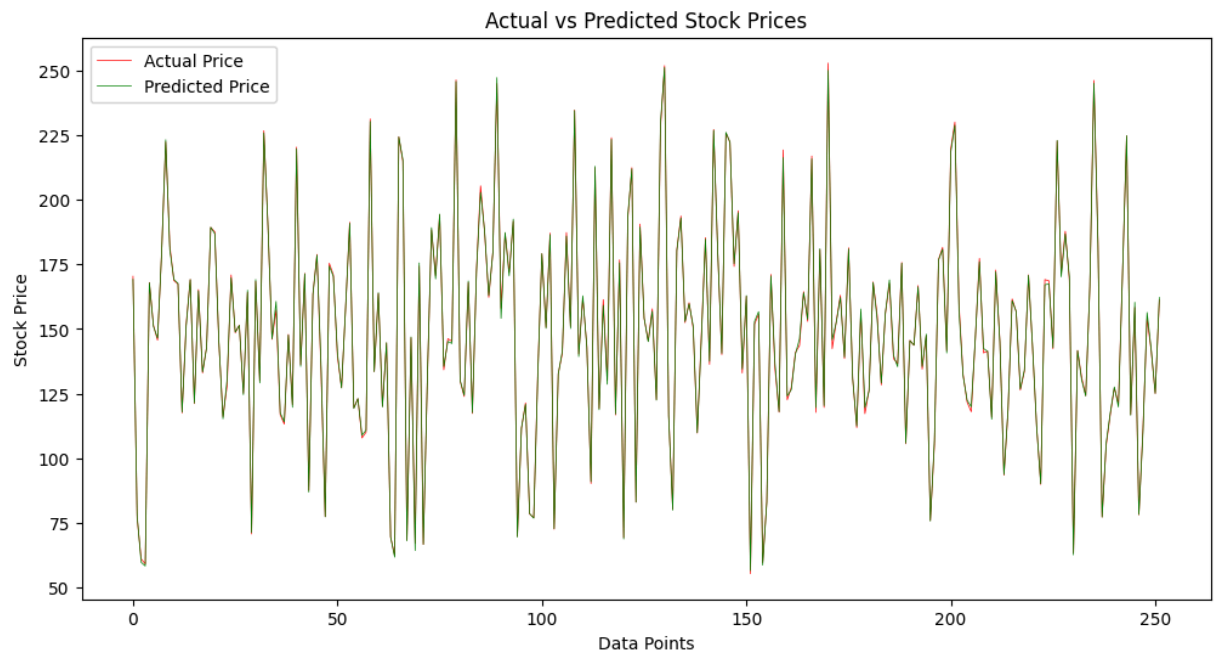
Visuaklize the datasets

In [44]: `import matplotlib.pyplot as plt`

In [78]: `plt.figure(figsize=(12,6))`

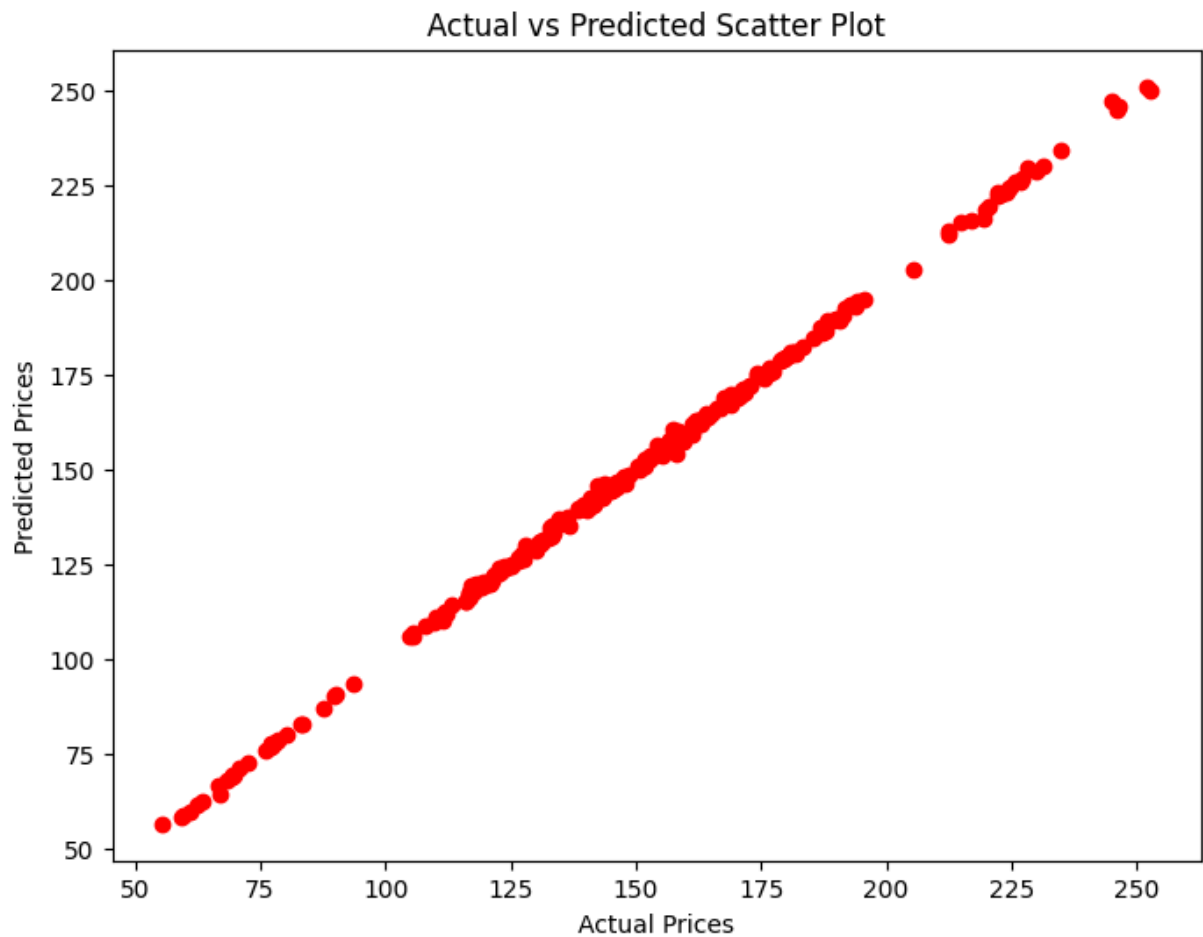
`plt.plot(y_test.values,label="Actual Price",linewidth=0.5,color="red")`
`plt.plot(y_pred,label="Predicted Price",linewidth=0.5,color="green")`

`plt.title("Actual vs Predicted Stock Prices")`
`plt.xlabel("Data Points")`
`plt.ylabel("Stock Price")`
`plt.legend()`
`plt.show()`



Show the scatter [plots

```
In [79]: plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred,color='red')
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Scatter Plot")
plt.show()
```



Predict the New datas

```
In [80]: new_data = pd.DataFrame({"Open": [210], "High": [215], "Low": [205], "Volume": [500000]})  
  
new_prediction = model.predict(new_data)  
  
print("\nNEW STOCK PRICE PREDICTION")  
print(new_prediction)
```

```
NEW STOCK PRICE PREDICTION  
[210.40142526]
```

```
In [81]: import os  
os.getcwd()
```

```
Out[81]: 'R:\\\\adarsa\\365\\stock_data'
```

```
In [ ]:
```